

1 DIRSIG BATCH DRIVER — DEVELOPMENT PLAN

1.1 CONTEXT

This project operates in the Space Domain Awareness (SDA) domain — the detection, tracking, and characterization of Resident Space Objects (RSOs) using space-based and ground-based EO sensors. The simulation chain being built generates synthetic imagery of RSOs against an Earth background, providing physics-grounded training data for AI/ML detection and characterization algorithms.

The goal is a Python-based batch simulation driver that programmatically generates DIRSIG5 input files and invokes `dirsig5` to produce synthetic EO imagery. The primary near-term output is an Earth background canvas in SWIR for use in a synthetic RSO dataset generator. Longer term, the driver will support full parametric sweeps over RSO orbit, attitude, sensor configuration, and atmospheric conditions.

The driver is built on `dirfm`, a Python library developed by Rendered.ai in collaboration with RIT's DIRS Lab that generates DIRSIG input files programmatically. Where `dirfm` does not cover SSA-specific requirements, supplementary modules fill the gap. The Jupyter notebook is the presentation and execution layer — it is both a development artifact and a handoff document.

You are not writing a GUI replacement. You are writing an orchestration layer that sits above `dirfm` and below `dirsig5`.

1.2 PHASE 0 — ENVIRONMENT SETUP AND DIRFM BASELINE

Clone `dirfm` from RIT's GitLab and `dirsig-channel` from Rendered.ai's GitHub. Install `dirfm` into the project conda environment. The objective of this phase is to get `dirfm` running against existing, relevant RIT demo scenarios — confirming the library works in your environment and that you understand how it generates the DIRSIG input file set before writing any new code.

Work through the following demos in order, using `dirfm` to reproduce each programmatically and running the result through `dirsig5`:

1. Ssa2 — static orbital scene; introduction to the file structure and radiometry pipeline
2. FlexibleMotion1 — FlexMotion model fundamentals
3. Ssa1 — SGP4-driven RSO, ground-to-space; the primary reference scenario for this project

The remaining demos (Ssa3, LightCurve1, EarthClouds1) are deferred to a later phase when the core driver is established. For each demo completed, read every input file and understand which `dirfm` API call produced it. The `dirsig-channel` graph examples are the reference for how `dirfm` is intended to be used — cross-reference them against the RIT demo file sets.

1.3 PHASE 0A — BASELINE NOTEBOOK

Produce a Jupyter notebook that documents and exercises the Phase 0 work. The notebook serves two purposes: it is a living record of what was learned about dirfm during the baseline phase, and it is the foundation that later phases will extend rather than replace.

Organize it as three sections corresponding to the three demos — one section per scenario. Each section should programmatically invoke dirfm to generate the input file set, run dirsigs, and display a representative output frame using spectral (SPy) for ENVI raster ingestion. Include brief narrative in markdown cells explaining what each demo demonstrates and which dirfm API calls are doing the work.

The notebook should be runnable end-to-end in a single kernel session without manual intervention.

1.4 PHASE 1 — DIRFM CAPABILITY ASSESSMENT

dirfm was developed in the context of Rendered.ai's platform — airborne sensors, visible and VNIR bands, urban and agricultural scenes. Its coverage of SSA-specific capabilities cannot be assumed. This phase answers two questions that gate the rest of the architecture.

Question 1: FlexMotion coverage. The SSA motion model requires SGP4 propagation from TLE inputs, ECEF coordinate positioning, and quaternion attitude time-series. These are non-negotiable for RSO simulation. To answer this, read the dirfm source modules responsible for motion file generation and attempt to configure an Ssa1-equivalent motion description using only the dirfm API. If the API exposes the required engines, the gap is closed. If it does not, the gap is characterized precisely enough to scope the supplementary motion module in Phase 2.

Question 2: SWIR band support. Attempt to configure a SWIR bandpass sensor and atmosphere profile using dirfm's sensor and atmosphere APIs. The question is whether the library's abstractions accommodate arbitrary spectral bandpass or are implicitly constrained to visible/VNIR. A successful DIRSIG run producing physically plausible SWIR output closes this question.

The output of this phase is a written gap report — what dirfm covers, what it does not, and what needs to be built. This report drives the scope of Phases 2 and 3.

1.5 PHASE 2 — MOTION MODULE

If Phase 1 confirms a gap in FlexMotion coverage, implement a FlexMotionWriter module that serializes the SSA-relevant motion model combinations to .motion XML. The minimum required engines are SGP4 (TLE input), ECEF waypoints, and quaternion attitude time-series. Validate output XML against the Ssa1 demo .motion file as ground truth — structurally identical output for equivalent inputs is the acceptance criterion.

This module is intentionally narrow. It generates .motion files and nothing else. No dirfm functionality is duplicated.

1.6 PHASE 3—ORCHESTRATION LAYER

Design a DirsigScenario dataclass that holds the complete parameter set for a single simulation run — RSO orbit, attitude, sensor configuration, atmosphere profile, collection time window, and output path. Implement two methods: build(), which calls dirfm APIs and the Phase 2 motion module to populate a run directory with the complete input file set; and run(), which invokes dirsigs5 via subprocess with the run directory as the working directory.

The scope of this layer is orchestration only — no physics logic, no file format knowledge beyond what dirfm and the motion module already encapsulate. The DirsigScenario is the single object a notebook cell instantiates and executes.

1.7 PHASE 4—NOTEBOOK

Assemble the complete workflow into a Jupyter notebook organized as independently re-runnable sections: environment validation, a single reference run reproducing Ssa1 programmatically via DirsigScenario, a small parameter sweep (3 orbits × 2 atmospheres), and output inspection using spectral (SPy) for ENVI raster ingestion and display.

The notebook is the primary handoff artifact. Write it so that a new team member with DIRSIG familiarity can follow the chain from parameters to pixels without additional documentation.